



PROYECTO FINAL





Estructura del proyecto

FASE 1 — Análisis Forense e Incidente SSH 1.1

Servicios comprometidos y vector de acceso 1.2

Auditoría del sistema 1.3 Contención y cierre del

exploit 1.4 Fortalecimiento de configuraciones 1.5

Recomendaciones · 1.6 Línea de tiempo

FASE 2 — Detección y Explotación RCE 2.1 Escaneo y

detección (Nmap) 2.2 Explotación paso a paso 2.3

Impacto y análisis 2.4 Medidas correctivas y

verificación

FASE 3 — Plan de Respuesta y SGSI ISO 27001 3.1

Plan de respuesta NIST SP 800-61 3.2 Mecanismos de

protección de datos 3.3 SGSI ISO 27001:2022 3.4

Presentación ejecutiva para gerencia

FASE 1 ANÁLISIS FORENSE E INCIDENTE DE SEGURIDAD

IDENTIFICACIÓN DEL VECTOR DE ACCESO · AUDITORÍA DEL SISTEMA · CONTENCIÓN DEL EXPLOIT

1



EVIDENCIA FORENSE: ACCESO SSH COMO ROOT

Ejecuté el comando `journalctl` para revisar los logs de SSH y encontré esta línea crítica: el 8 de octubre a las 17:40:59, alguien accedió como root desde la IP 192.168.0.134. Esto es muy grave porque root es el administrador del sistema. El atacante utilizó un ataque de fuerza bruta, probando muchas contraseñas hasta acertar con la débil que teníamos. Además, detecté el servicio FTP activo con una configuración muy peligrosa: permitía acceso anónimo, permitía conexiones de usuarios locales, y además permitía subir archivos. Esto significa que cualquier persona en la red podía conectarse sin contraseña y dejar archivos en el servidor.

AUDITORIA DEL SISTEMA DE INSPECCIÓN PRELIMINAR DEL SERVIDOR

Verificación 1 — LIMPIO

Comando:

bash

sudo ls -l /proc/*/exe 2>/dev/null | grep -E 'tmp|shm|var/run'

Resultado: Vacío

Explicación: Busca programas maliciosos ejecutándose desde carpetas temporales (/tmp, /dev/shm, /var/run), donde los atacantes suelen esconder malware. No se encontró nada.

Estado:  LIMPIO

Verificación 2 — LIMPIO

Comando:

bash

sudo grep ":0:" /etc/passwd

Resultado:

text

root:x:0:0:root:/root:/bin/bash

Explicación: Busca usuarios con privilegios de superusuario (UID 0). Solo aparece el root legítimo. El atacante no creó cuentas backdoor.

Estado:  LIMPIO

Verificación 3 — LIMPIO

Comando:

bash

sudo chkrootkit

Resultado: not infected / not found

Explicación: Busca rootkits (programas que se esconden en el sistema operativo para no ser detectados). El resultado indica que no hay malware oculto.

Estado:  LIMPIO

Verificación 4 — HALLAZGO

Herramienta: Autopsy

Análisis: /dev/sda1 → directorio /srv/ftp/

Resultado:

text

Modified: 2024-10-08 16:09:00 EDT

Explicación: El directorio fue modificado el 8 de octubre a las 16:09:00, que es 1 hora y 31 minutos antes del acceso SSH por fuerza bruta (17:40:59).
Esto indica actividad preparatoria del atacante, probablemente utilizando el servicio FTP anónimo para explorar o preparar el terreno antes del ataque principal.

Estado:  HALLAZGO (actividad previa al ataque)

Medida 1 — Detener SSH

Comando: `systemctl stop ssh`

Qué hace: Detiene el servicio SSH temporalmente

Por qué: Evitar que el atacante vuelva a conectarse mientras se analiza el sistema

Estado:  APLICADO

Medida 2 — Bloquear IP atacante

Comando: `ufw deny from 192.168.0.134`

Qué hace: Bloquea la IP del atacante en el firewall

Por qué: Impide cualquier conexión futura desde esa dirección IP maliciosa

Estado:  APLICADO

Medida 3 — Eliminar FTP

Comando: `apt purge vsftpd -y`

Qué hace: Elimina completamente el servidor FTP y su configuración

Por qué: El servicio tenía acceso anónimo y permitía subir archivos, era una puerta abierta

Estado:  APLICADO

Medida 4 — Endurecer firewall

Comando: `ufw default deny incoming; allow 22,80/tcp; ufw enable`

Qué hace: Configura el firewall para bloquear todo excepto SSH y web

Por qué: Reduce la superficie de ataque, solo deja abierto lo necesario

Estado:  APLICADO

Medida 5 — Deshabilitar root SSH

Comando: `echo "PermitRootLogin no" | sudo tee -a /etc/ssh/sshd_config`

Qué hace: Deshabilita el acceso SSH como usuario root

Por qué: Obliga a usar usuarios normales que luego escalan, dejando registro de quién accede

Estado:  APLICADO

Medida 6 — Cambiar contraseña root

Comando: `passwd root`

Qué hace: Cambia la contraseña del root por una robusta

Por qué: La contraseña anterior era débil y fue crackeada por fuerza bruta

Requisito: Mínimo 16 caracteres, mayúsculas, minúsculas, números y símbolos

Estado:  APLICADO

Medida 7 — Corregir WordPress y MySQL

Comando: `ALTER USER... + chmod 600 wp-config.php`

Qué hace: Cambia la contraseña de MySQL de '123456' a una segura y restringe permisos

Por qué: La contraseña por defecto era un riesgo crítico; el archivo no debe ser legible por cualquiera

Estado:  APLICADO

LÍNEA DE TIEMPO DEL INCIDENTE — 8 OCTUBRE 2024



LÍNEA DE TIEMPO DEL INCIDENTE

16:09:00 - Modificación /srv/ftp/ - Actividad preparatoria vía FTP anónimo

17:40:59 - Acceso SSH como root - Ataque de fuerza bruta exitoso desde 192.168.0.134

17:40:59 - Sesión interactiva root - Session opened for user root (journalctl)

Posterior - Acceso wp-config.php - Credencial 123456 detectada para usuario wordpress

RECOMENDACIONES CLAVE

- 1.Fail2ban - Bloqueo tras 3 intentos fallidos SSH*
- 2.Autenticación SSH por clave pública - Deshabilitar contraseñas*
- 3.Cambiar puerto SSH a no estándar (ejemplo 42222) - Evitar escaneos automáticos*
- 4.SIEM en tiempo real (Wazuh) - Alertas de acceso*

FASE 2 FASE 2 DETECCIÓN Y EXPLOTACIÓN DE VULNERABILIDAD

- RCE WORDPRESS TWENTY-FOUR · CVSS 9.8 CRÍTICO ·

REMOTE CODE EXECUTION

2



ESCANEEO NMAP Y VULNERABILIDAD. RECONOCIMIENTO: NMAP-SV-SC 192.168.1.33



Ejecuté Nmap para ver qué servicios estaban activos. Encontré el puerto 21 con FTP que permitía acceso anónimo, el puerto 22 con SSH, y el puerto 80 con Apache y WordPress.

La vulnerabilidad crítica que detecté fue en el archivo 404.php del tema Twenty Twenty-Four. Estaba vacío y accesible públicamente, lo que permitía inyectar código PHP malicioso. Esto es un RCE, ejecución remota de código, y es muy peligroso porque cualquier atacante puede ejecutar comandos en el servidor sin necesidad de autenticación.

Además, encontré directory listing activado en la carpeta de uploads, el tema desactualizado, y la versión de WordPress visible. Todo esto aumenta la superficie de ataque

CADENA DE ATAQUE — RCE A SHELL REVERSA

Primero, inyecté una web shell. El archivo 404.php estaba vacío, así que lo usé para crear un archivo shell.php con código PHP malicioso que ejecuta comandos del sistema.

Segundo, verifiqué que funcionaba. Ejecuté el comando id y me devolvió que era el usuario www-data. Confirmé que podía ejecutar comandos en el servidor.

Tercero, establecí una shell reversa. En lugar de enviar comandos uno por uno, abrí una terminal interactiva del servidor en mi máquina usando netcat. Ahora tenía control total sobre el servidor como usuario www-data.

Cuarto, intenté escalar privilegios a root. Busqué programas con permisos especiales y encontré que pkexec era vulnerable al exploit CVE-2021-4034. Ejecuté el exploit para intentar convertirme en root.



ANÁLISIS DE IMPACTO CIA + CORRECCIONES APLICADAS

Qué descubrí

Escaneando el servidor con Nmap, encontré que el tema de WordPress Twenty Twenty-Four tenía el archivo 404.php vacío y accesible públicamente. Esto permitía inyectar código PHP malicioso.

Qué hice

1. Inyecté una web shell para ejecutar comandos.
2. Verifiqué que funcionaba con el comando id (respondió www-data).
3. Establecí una shell reversa con netcat para tener control total del servidor.
4. Intenté escalar a root usando el exploit CVE-2021-4034 (pkexec).

Impacto (matriz CIA)

- Confidencialidad ALTA: podía leer wp-config.php, /etc/passwd, logs.
- Integridad ALTA: podía modificar o borrar archivos.
- Disponibilidad MEDIA: podía tumbar el sitio web.
- Alcance CRÍTICO: podía atacar la red interna.

Cómo lo corregí

- Eliminé shell.php y restauré 404.php.
- Desactivé el listado de directorios.
- Puse permisos seguros (chmod 600 wp-config.php).
- Bloqueé ejecución de PHP en la carpeta uploads.
- Actualicé el tema a la última versión.
- Verifiqué que la vulnerabilidad ya no funcionaba (curl devolvió 404/403).

Recomendaciones adicionales

- Instalar WAF ModSecurity.
- Activar actualizaciones automáticas de WordPress.
- Deshabilitar XML-RPC en .htaccess.



SECCIÓN FASE 3 FASE 3 PLAN DE RESPUESTA A INCIDENTES Y SGSI ISO 27001 NIST SP 800-61 REV. 2 · ISO/IEC 27001:2022 · PLAYBOOKS · PRESENTACIÓN EJECUTIVA

3



CICLO DE RESPUESTA A INCIDENTES

El plan de respuesta a incidentes sigue el modelo NIST de 4 fases.

En la Fase 1, Preparación, nos anticipamos al ataque. Creamos un equipo CSIRT con 5 roles, instalamos herramientas como Wazuh y Suricata, configuramos Fail2ban y autenticación por clave pública, y establecimos backups cifrados fuera del sitio.

En la Fase 2, Detección y Análisis, clasificamos los incidentes por gravedad: P1 crítica en menos de 1 hora, P2 alta en menos de 4 horas, P3 media en 24 horas, P4 baja en 72 horas. Vigilamos indicadores como logins root externos o archivos php en uploads. El objetivo es detectar en menos de 30 minutos.

En la Fase 3, Contención y Erradicación, tenemos playbooks específicos. Para un ataque de fuerza bruta SSH, seguimos 10 pasos. Para un RCE en WordPress, seguimos 8 pasos. Aplicamos firewall restrictivo, eliminamos servicios inseguros, y restauramos archivos limpios.

En la Fase 4, Post-Incidente, hacemos un informe obligatorio para incidentes graves, medimos el tiempo de respuesta buscando menos de 2 horas, y aseguramos que el mismo incidente no vuelva a ocurrir con mejora continua.*



PROCEDIMIENTOS DE RESPUESTA PROBADOS Y VALIDADOS

El Playbook A es para ataques de fuerza bruta SSH. Los pasos clave son: detener SSH para que el atacante no siga entrando, bloquear su IP, cambiar la contraseña de root, deshabilitar el acceso root por SSH, eliminar el FTP que era una puerta abierta, corregir las credenciales de MySQL, endurecer el firewall, y verificar con chkrootkit que no quede malware.

El Playbook B es para ataques RCE en WordPress. Los pasos clave son: aislar el servidor web deteniendo Apache, eliminar la web shell maliciosa, restaurar el archivo 404.php original, desactivar el listado de directorios, poner permisos seguros en especial el archivo wp-config.php, bloquear la ejecución de PHP en la carpeta uploads, actualizar el tema vulnerable, y revisar que no haya usuarios creados por el atacante en WordPress o MySQL.



MATRIZ DE RIESGOS — METODOLOGÍA CUALITATIVA (ESCALA 1-5)

Amenaza | Activo | P | I | Riesgo | Control ISO 27002 | Acción Fuerza bruta SSH | Servidor SSH | 5 | 5 | 25 CRÍTICO | A.8.5 | Fail2ban + clave pública RCE WordPress | Servidor web | 4 | 5 | 20 ALTO | A.8.8 | WAF + actualizaciones FTP acceso anónimo | Servidor FTP | 4 | 4 | 16 ALTO | A.8.21 | apt purge vsftpd Inyección SQL | MySQL / WP | 3 | 5 | 15 ALTO | A.8.28 | WAF + auditoría código Escalada SUID | Sistema OS | 3 | 5 | 15 ALTO | A.8.18 | Auditoría SUID mensual Pérdida datos | Todos | 2 | 5 | 10 MEDIO | A.8.13 | Backups AES-256 diarios Phishing | Credenciales | 3 | 3 | 9 MEDIO | A.6.3 | Formación + simulaciones DDoS | Disponibilidad | 2 | 3 | 6 BAJO | — | CDN / antiDDoS

QUÉ ES LA MATRIZ DE RIESGOS

Es una tabla que evalúa cada amenaza según:

- P = Probabilidad (del 1 al 5, qué tan probable es que ocurra)*
- I = Impacto (del 1 al 5, qué tan grave sería si ocurre)*
- Riesgo = P x I (multiplicamos probabilidad por impacto)*

Cuanto más alto el número, más grave es el riesgo.

CONTROLES ANEXO A + KPIS TÍTULO: CONTROLES ISO

27001:2022 ANEXO A + KPIS DEL SGSI

Controles implementados:

A.5.1 | Política de seguridad | Documento aprobado por Dirección

A.5.24 | Gestión de incidentes | Playbooks A y B documentados

A.6.3 | Formación y concienciación | Programa anual para todo el personal

A.8.2 | Acceso privilegiado | PermitRootLogin no; sudo auditado

A.8.5 | Autenticación segura | MFA SSH, WP admin, MySQL externo

A.8.8 | Gestión de vulnerabilidades | OpenVAS semanal; parches < 30 días

A.8.13 | Copias de seguridad | AES-256 offsite + pruebas mensuales A.8.16 | Monitorización | Wazuh SIEM + alertas en tiempo real

A.8.28 | Codificación segura | WAF ModSecurity + OWASP Top 10

KPIs del SGSI:

MTTD → < 30 min

MTTR → < 2h (P1/P2)

Recurrencia → 0%

Parches críticos → 100% en < 30 días

Personal formado → 100%

Backups validados → 100%

Vulns. abiertas > 30d → 0

ROADMAP HACIA CERTIFICACIÓN ISO 27001

s un plan de 12 meses para mejorar la seguridad paso a paso y conseguir la certificación ISO 27001. Cada dos meses se trabaja en un área diferente.

Mes 1-2 | Fundamentos Fail2ban · MFA · Backups AES-256 · Purga vsftpd · Permisos WP

Mes 3-4 | Monitorización SIEM Wazuh · IDS Suricata · Dashboards de alertas en tiempo real

Mes 5-6 | Aplicación web WAF ModSecurity · Actualizaciones automáticas · HTTPS/HSTS

Mes 7-8 | Personas Formación de personal · Simulacros phishing · Política escritorio limpio

Mes 9-10 | SGSI Documentación completa · Auditoría interna · Análisis de brechas ISO 27001

Mes 11-12 | Certificación Pre-auditoría externa · Resolución de no conformidades · Solicitud ISO 27001

CONCLUSIONES Y RECOMENDACIONES FINALES

- *Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.*
- *Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.*
- *Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.*

2 LAS CONTRASEÑAS
DEBEN SER SEGURAS

